Group 04

SmartSpend AI
Use-Case Specification

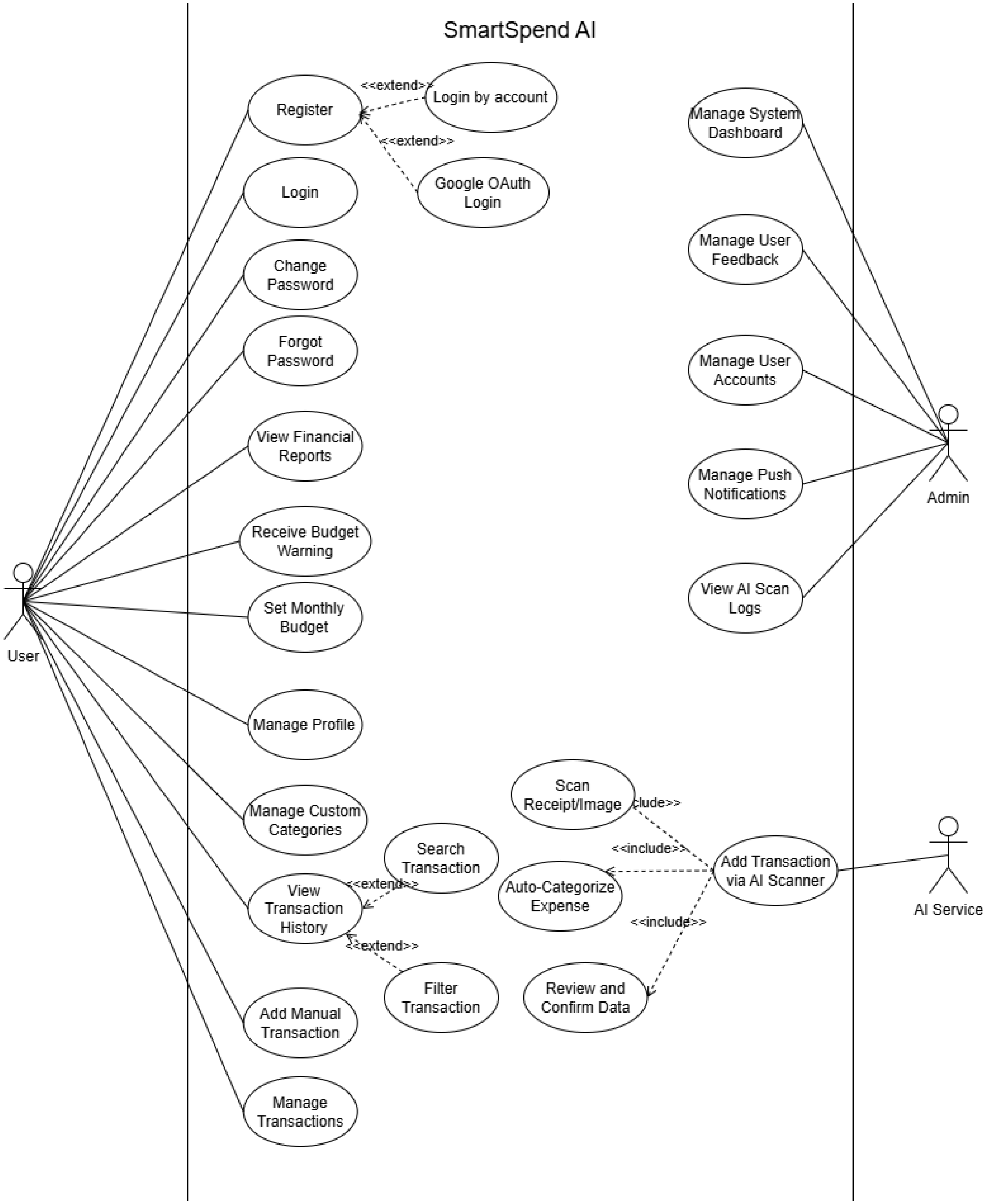
Version 1.0

Revision History

Date	Version	Description	Author
10/07/26	1.0	Initial draft for SmartSpend AI core use cases	Tạ Bỉnh Phong Phan Văn Bá Đạt Nguyễn Trịnh Tuấn Văn Nguyễn Trần Lan Viên

Table of Contents

1. Use-case Model



2. Use-case Specifications

2.1 Use-case: Register

Use case Name	Register new account
Brief description	This use case describes how the User create new account
Actors	User
Basic Flow	1. At the "Login" page, the user presses the "Register now" button 2. User enters "name" in name textbox 3. User enters "email" in email textbox 4. User enters "password" in password textbox 5. User clicks "Register" button 6. System change to "OTP" page and send verification mail to user's email 7. User opens mail, remember "OTP" code to verify 8. User return to app, rewrite OTP code to OTP page, then press "Enter" button 9. System change to "Get_user-profile" page 10. User enters "full name", "age", "job", "income" in each textbox. Then the user clicks "save" button to save profile 11. System returns to homepage with new account has been registered
Alternative Flows	**Alternative flow 1: User's register with Google** 1. At the "Login" page, the user presses the "Login with Google" button 2. User chooses 1 Google account to login with 3. User enters "Google password" into password textbox, then clicks continue 4. System change to "Get_user-profile" page 5. User enters "full name", "age", "job", "income" in each textbox. Then the user clicks "save" button to save profile 6. System returns to homepage with new account has been registered **Alternative flow 2: User's name is invalid** 1. From #2 of the basic flow, system pop-up error that user's name is invalid (eg: blank, too long) 2. Return to #2 to change new name **Alternative flow 3: User enter wrong email** 1. From #5 of the basic flow, the system displays a pop-up error indicating that the email address is invalid 2. Return to #3 to correct user's email

	Alternative flow 4: User's password is not follow system's condition 1. From #4 of the basic flow, system pop-up error that password is not accepted (eg: missing number, uppercase letter, special character, blank) 2. Return to #4 to change the password **Alternative flow 5: User did not get the right OPT code in time** 1. From #8 of the basic flow, after 1 minute without verification, the system expires the register's OTP 2. Return to #6, press "Resend OTP" button to get another OTP code **Alternative flow 6: User's profile is invalid** 1. From #10 of the basic flow, system pop-up error for each invalid information (eg: full name is blank - birthday too old - job has not been chosen) 2. Return to #10, rewrite/rechoice invalid information
Pre-conditions	User goes to register page
Post-conditions	The user successfully registered

2.2 Use-case: Login

Use case Name	Login account
Brief description	This use case describes how the User login account
Actors	User
Basic Flow	1. At the "Login" page 2. User enters "email" in email textbox 3. User enters "password" in password textbox 4. User clicks "Login" button 5. System returns to homepage with new user account has logged in
Alternative Flows	**Alternative flow 1: User login with Google** 1. At the "Login" page, the user clicks "Login with Google" button 2. User chooses 1 Google account to login with 3. User enters "Google password" into password textbox, then clicks continue 4. System returns to homepage with new Google account has logged in **Alternative flow 2: User enter wrong email** 1. From #4 of the basic flow, system pop-up error that email or password are incorrect

	2. Return to #2 to correct email **Alternative flow 3: User enter wrong password** 1. From #4 of the basic flow, system pop-up error that email or password are incorrect 2. Return to #2 to correct password
Pre-conditions	User goes to login page
Post-conditions	The user successfully logged in

2.3 Use-case: Forgot Password

Use case Name	Forgot Password
Brief description	This use case describes how the user reset their password
Actors	User
Basic Flow	1. At the login page or register page, user chooses the “forgot password” button 2. User enters “email” in recover email textbox, then presses “send OTP code" button 3. System changes to “OTP” page and send verification mail to user’s email 4. User opens mail, remember “OTP” code to verify 5. User returns to app, rewrite OTP code to OTP page, then presses “continue” button 6. User enters “new password” in new password textbox 7. User clicks "confirm" button 8. System pop-up “successful”, then returns to Login page
Alternative Flows	**Alternative flow 1: User enter wrong email** 1. From #2 of the basic flow, system pop-up error that can not find user’s email 2. Return to #2 to correct user’s email **Alternative flow 2: User did not get the right OPT code in time** 1. From #5 of the basic flow, after 1 minute without verification, system expires register’s OTP 2. Return to #3, presses “send OTP again” button to get another OTP code **Alternative flow 3: User’s new password does not meet the system requirements** 1. From #6 of the basic flow, system pop-up error that the password is invalid (eg: missing number, uppercase letter, special character, blank)

	2. Return to #6 to change the password
Pre-conditions	1. The user have registered an account 2. The user's account status is active.
Post-conditions	The user successfully reset their password

2.4 Use-case: Manage Profile

Use case Name	Manage profile
Brief description	This use case describes how the User change profile
Actors	User
Basic Flow	1. At the “Profile” page, user chooses the “change personal information” button 2. User can change “profile picture” by clicking to old profile picture, then choosing an image from picture gallery or taking a photo 3. User can enter “new name” in name text box 4. User can enter “new birthday” in birthday textbox 5. User clicks "save changes" button 6. System pop-up “successful”, then returns to Profile page
Alternative Flows	**Alternative flow 1: System requires Gallery-access permission (First-time use)** 1. From #2 of the basic flow, System detects Gallery-access permission is not configured and displays a permission request dialog. 2. User click Allow” 3. Return to #2 to continue **Alternative flow 2: User deny Gallery-access permission** 1. From #2 of the basic flow, System detects Gallery-access permission is not configured and displays a permission request dialog. 2. User clicks “Don’t allow” 3. System displays an error message: “Gallery access is required to select photos. Please enable it in Settings or use the Camera.” 4. User acknowledges the error message. 5. Return to #2 to continue **Alternative flow 3: System requires Camera permission (First-time use)** 1. From #2 of the basic flow, System detects Camera permission is not configured and displays a permission request dialog. 2. User click “Allow” 3. Return to #2 to continue

	Alternative flow 4: User deny Camera permission 1. From #2 of the basic flow, System detects Camera permission is not configured and displays a permission request dialog. 2. User click “Don’t allow” 3. System displays an error message: “Camera is required to take photos. Please enable it in Settings or choose from the Gallery.” 4. User acknowledges the error message. 5. Return to #2 to continue **Alternative flow 5: User choose wrong file for profile picture** 1. From #2 of the basic flow, system pop-up error of wrong type of file can be used 2. Return to #2 to choose another profile picture **Alternative flow 6: User choose wrong file for profile picture** 1. From #2 of the basic flow, system pop-up error of wrong type of file can be used 2. Return to #2 to choose another profile picture **Alternative flow 7: User’s new name is invalid** 1. From #3 of the basic flow, system pop-up error that user’s new name is invalid(eg: blank, too long) 2. Return to #3 to change new name **Alternative flow 8: User’s new birthday is invalid** 1. From #4 of the basic flow, system pop-up error that user’s new birthday is invalid (eg: too old, future date) 2. Return to #4 to correct birthday
Pre-conditions	1. The user have logged in an account 2. The user's account status is active.
Post-conditions	The user successfully changed their personal information

2.5 Use-case: Change password

Use case Name	Change password
Brief description	This use case describes how the User change password
Actors	User
Basic Flow	1. At the “Profile” page, user chooses the “change password” button 2. User enters “old password” in old-password textbox

	3. User enters "new password" in new-password textbox 4. User clicks "Change password" button 5. System pop-up "successful", then return to Profile page
Alternative Flows	**Alternative flow 1: User enter wrong old-password when change password** 1. From #2.4 of the basic flow, system pop-up error that old password is incorrect 2. Return to #2.2 to change old-password **Alternative flow 2: User's new password does not meet the system requirements** 1. From #2.3 of the basic flow, system pop-up error that password is invalid (eg: missing number, uppercase letter, special character, blank) 2. Return to #2.3 to change password
Pre-conditions	1. The user have logged in an account 2. The user's account status is active.
Post-conditions	The user successfully changed personal information

2.6 Use-case: Manage Custom Categories

Use case Name	Manage Custom Categories
Brief description	This use case describes how the User can perform CRUD (Create, Read, Update, Delete) operations on their personalized expense/income categories to tailor the financial tracking experience.
Actors	User
Basic Flow	1. From the main screen, the user navigates to the "Category Management" section. 2. The system retrieves and displays a list of all existing custom categories created by the user, alongside the system's default categories (Read). 3. The user clicks on the "Add New Category" button. 4. The system displays a form requiring the user to enter the Category Name and select a color code for personalization. 5. The user fills in the valid details and clicks the "Save" button (Create). 6. The system validates the inputs, ensures the category name is unique within the user's account, and records the new category into the database. 7. The system updates the list and displays the category added successfully.
Alternative Flows	**Alternative flow 1: Update an existing custom category (Update)** 1. At step #3 of the basic flow, instead of creating a new one, the user selects an existing custom category and clicks the "Edit" button. 2. The system populates the form with the current details of that category.

	3. The user modifies the name, icon, or color and clicks "Save". 4. The system validates, updates the database, and displays a success message. **Alternative flow 2: Delete a custom category (Delete)** 1. At step #3 of the basic flow, the user selects a custom category and clicks the "Delete" icon. 2. The system triggers a confirmation dialog: *"Are you sure you want to delete this category? All transactions mapped to this category will be moved to 'Uncategorized'."* - *Scenario 2a: User clicks "Confirm":* The system updates the transactions, removes the category from the database, and refreshes the list. - *Scenario 2b: User clicks "Cancel":* The system closes the dialog, preserving the category data. **Alternative flow 3: Duplicate category name** 1. At step #6 of the basic flow, if the system detects that the user entered a name that already exists in their custom list. 2. The system rejects the action and displays an error message: *"This category name already exists. Please choose a unique name."* 3. The form remains open for the user to change the input. **Alternative flow 4: System fails to update data** 1. From #5 of the basic flow, System cannot write data to the server due to a connection timeout. 2. System displays an error notification: "Unable save the category." and provides two options: 'Retry' or 'Cancel'. 3. If Admin chooses 'Retry': Continue step #5 in the basic flow. 4. If Admin chooses 'Cancel': System discards the changes. Continue step #2 in the basic flow.
Pre-conditions	The user is successfully logged into the Smart Spend AI application.
Post-conditions	The user's personalized category list is successfully updated in the database, immediately becoming available as options in the Auto-Categorize Expense and Set Monthly Budget features.

2.7 Use-case: Add Manual Transaction

Use case Name	Add Manual Transaction.
Brief description	This use case describes how the User can manually enter and save a new spending or income transaction without using the AI scanning feature.
Actors	User
Basic Flow	1. From the main dashboard, the user clicks on the record icon at the bottom centre, choose the manual input.

	2. The system displays a blank form requiring the user to fill out the following information: - Transaction Name - Transaction Type (Income or Expense) - Amount (Required) - Date & Time (Defaults to current date/time) - Category (Dropdown list selecting from default or custom categories) - Note/Description (Optional) 3. The user fills in the valid details and clicks the "Save" button. 4. The system validates the entered data, records the new transaction entry into the database, and calculates the impact on the wallet balance. 5. The system returns the user to the previous screen and displays a message: *"Transaction added successfully!"*
Alternative Flows	**Alternative flow 1: User enters an invalid or empty amount** 1. At step #3 of the basic flow, if the user leaves the "Amount" field blank or inputs non-numeric/negative values. 2. The system rejects the submission and highlights the field with an error message: *"Amount is a required field and must be greater than 0."* 3. The system keeps the form open for the user to correct the input (Go back to step #3). **Alternative flow 2: Cancel transaction creation** 1. At any point before clicking "Save", the user clicks the "Back" or "Cancel" button. 2. The system closes the blank form without saving any data and returns the user to the previous screen. **Alternative flow 4: System fails to update data** 1. From #3 of the basic flow, System cannot write data to the server due to a connection timeout. 2. System displays an error notification: "Unable save the category." and provides two options: 'Retry' or 'Cancel'. 3. If Admin chooses 'Retry': Continue step #3 in the basic flow. 4. If Admin chooses 'Cancel': System discards the changes. Continue step #2 in the basic flow.
Pre-conditions	1. The user is successfully logged into the Smart Spend AI application. 2. The user is on the main dashboard interface or transaction list screen.
Post-conditions	The transaction is successfully stored in the user's ledger, and the system immediately updates the category breakdown chart (*View Financial Reports*) and the remaining allowance (*Set Monthly Budget*).

2.8 Use-case: Manage Transactions

Use case Name	Manage Transactions
Brief description	This use case describes how the User can manage their recorded financial activities, including viewing the complete transaction list, editing transaction details, or deleting incorrect entries.
Actors	User
Basic Flow	1. From the main dashboard, the user navigates to the "Transactions" 2. The system retrieves and displays a comprehensive list of all past income and expense transactions, sorted chronologically from newest to oldest. 3. The user selects a specific transaction from the list to view its details and clicks the "Edit" button. 4. The system populates the transaction form with the existing data (Name, Type, Amount, Date, Category, and Notes). 5. The user updates the necessary fields and clicks the "Save Changes" button. 6. The system validates the modified data, overrides the old record in the database, and recalculates the wallet balance. 7. The system returns the user to the transaction list screen and displays a success notification: "Transaction updated successfully."
Alternative Flows	**Alternative flow 1: Delete a transaction (Delete)** 1. At step #3 of the basic flow, instead of editing, the user clicks the "Delete" (Trash) icon on the selected transaction. 2. The system displays a confirmation pop-up: *"Are you sure you want to permanently delete this transaction? This action cannot be undone."* - *Scenario 2a: User clicks "Confirm":* The system deletes the transaction from the database, restores the corresponding amount to the user's total balance, and refreshes the transaction list. - *Scenario 2b: User clicks "Cancel":* The system closes the pop-up and preserves the transaction data. **Alternative flow 2: User enters invalid data during editing** 1. At step #5 of the basic flow, if the user modifies the amount to an invalid value (e.g., zero, negative number, or empty field). 2. The system rejects the submission, highlights the error field, and alerts the user: *"Amount must be a positive number greater than 0."* 3. The system remains on the edit screen until the input is corrected. **Alternative flow 3: Cancel modification** 1. At any point before saving changes, the user clicks the "Cancel" or "Back" button. 2. The system discards all modifications and returns the user safely to the

	main transaction ledger.
Pre-conditions	1. The user is successfully logged into the Smart Spend AI application. 2. The user has at least one recorded transaction in their database history.
Post-conditions	The transaction ledger is updated accurately in the database, immediately triggering recalculations in the *View Financial Reports* (charts) and *Set Monthly Budget* (allowance progress).

2.9 Use-case: View Transaction History

Use case Name	View Transaction History.
Brief description	This use case describes how the **User** can review their past financial activities through a chronological ledger, with the ability to search for specific entries or filter transactions by various criteria (such as date range or category).
Actors	User
Basic Flow	1. From the main dashboard, the user selects the “Transaction History” option from the navigation menu. 2. The system retrieves the user's transaction records from the database. 3. The system displays a comprehensive list of all transactions, automatically sorted in reverse chronological order (newest items on top) with summary indicators (total income vs. total expense for the current period). 4. The user scrolls through the list to review individual entry summaries (including Name, Amount, and Category Tag).
Alternative Flows	**Alternative flow 1: User searches for a specific transaction (<<extend>> Search Transaction)** 1. At step #3 of the basic flow, the user taps on the Search bar and enters keywords (e.g., merchant name, item description, or amount). 2. As the user types, the system filters the list in real-time to display only transactions containing the specified keywords. 3. If no matching records are found, the system displays an empty state illustration with the message: “No transactions match your search keywords.” **Alternative flow 2: User filters transactions by custom criteria (<<extend>> Filter Transaction)** 1. At step #3 of the basic flow, the user clicks on the “Filter” icon. 2. The system opens a filter panel offering criteria such as: * Date Range (e.g., Today, This Week, Last 30 Days, Custom Range) * Transaction Type (Income vs. Expense) * Category (selecting specific default or custom categories)

	3. The user selects their desired criteria and clicks “Apply Filter”. 4. The system re-calculates the totals and updates the screen to render only the transactions satisfying the active filters. **Alternative flow 3: The user has no transaction records** 1. At step #2 of the basic flow, if the database indicates this is a brand new account with zero logged transactions. 2. The system bypasses step #3 and renders a friendly onboarding screen saying: “You haven't added any transactions yet. Tap the record button below to get started!”
Pre-conditions	1. The user is successfully logged into the Smart Spend AI application. 2. The user is on the main application dashboard.
Post-conditions	The system safely presents the requested transaction historical view without altering any underlying financial data entries in the database.

2.10 Use-case: Set Monthly Budget

Use case Name	Set Monthly Budget.
Brief description	This use case describes how the User can set up or adjust a spending limit for a specific month.
Actors	User
Basic Flow	1. From the main screen, the user selects the “Budget” (Budget Management) feature. 2. The system displays the current month's budget along with a chart and the budget for each category. 3. The user clicks to edit or delete the budget. 4. The system displays the edit form. 5. The user enters a valid spending limit amount for each category and clicks the “Save” button. 6. The system validates the data and records the new budget configuration into the database. 7. The system displays and updates the budget progress on the management screen.
Alternative Flows	**Alternative flow 1: User chooses to delete the budget** 1. At step #3 of the basic flow, if the user selects the “Delete” option. 2. The system displays a confirmation pop-up: *“Are you sure you want to delete this budget?”* 3. The user clicks “Confirm” 4. The system removes the budget data from the database, clears the chart, and updates the management screen to show no active budget. (Use case ends here).

	Alternative flow 2: User enters an invalid budget amount 1. At step #5 of the basic flow, if the user enters an invalid amount (e.g., negative number or non-numeric characters) for any category. 2. The system displays an error message: *"Please enter a valid positive amount for the categories"* 3. The system keeps the form open and prompts the user to correct the input (Go back to step #5). **Alternative flow 4: System fails to update data** 1. From #5 of the basic flow, System cannot write data to the server due to a connection timeout. 2. System displays an error notification: "Unable save the category" and provides two options: 'Retry' or 'Cancel'. 3. If Admin chooses 'Retry': Continue step #5 in the basic flow. 4. If Admin chooses 'Cancel': System discards the changes. Continue step #2 in the basic flow.
Pre-conditions	1. The user is successfully logged into the Smart Spend AI application. 2. The user's account status is active.
Post-conditions	The monthly budget threshold for categories is successfully updated or deleted in the database, directly reflecting on the budget charts and the *Receive Budget Warning* trigger.

2.11 Use-case: Receive Budget Warning

Use case Name	Receive budget warning
Brief description	This use case describes how the User get a warning from reaching monthly budget
Actors	User
Basic Flow	1. At the "receipt scanning" page, after the user has done the save transaction, user's expenses has over the limit 2. System pop-up a warning notification that the user's expenses is nearly over the limit: "Your expenses have nearly reached your budget, please look out for it" 3. User reads the warning notification and close it by press "x" button
Alternative Flows	**Alter 1: User get warning notification after set expense in information page** 1. At the "Information" page, after the user has done the save transaction, user's expenses has 2. System pop-up a warning notification that the user's expenses is nearly over the limit: "Your expenses have nearly reached your budget, please look out for it" 3. User reads the warning notification and close it by press "x" to button

Pre-conditions	1. The user is successfully logged into the Smart Spend AI application. 2. The user's account status is active. 3. The user's expense is over the limit.
Post-conditions	The user receives a warning notification

2.12 Use-case: View Financial Reports

Use case Name	View financial reports
Brief description	This use case describes how the User can view their financial report
Actors	User
Basic Flow	1. From the homepage, the user selects the "Finance Report" feature. 2. System displays the current month's budget, month's expense 3. System displays a circle chart and a multi-line chart for categories with week units. 4. System displays user's expense by category in week units
Alternative Flows	**N/A**
Pre-conditions	1. The user is successfully logged into the Smart Spend AI application. 2. The user's account status is active. 3. The user has at least one recorded transaction in their database history.
Post-conditions	The user can see their financial report and keep track of their expense

2.13 Use-case: Add Transaction via AI Scanner

Use case Name	Add Transaction via AI Scanner
Brief description	This use case describes how the User can add a new expense transaction by scanning a receipt using AI and OCR technology.
Actors	User
Basic Flow	1. User clicks on "Add Transaction via AI" button on the homepage 2. The system checks permissions and displays the Receipt Scanning Screen (which includes a camera viewfinder and a "Choose from Gallery" button). 3. The user aligns the receipt within the viewfinder and clicks the "Capture" button. 4. The system performs Optical Character Recognition (OCR) on the receipt image and extracts information (Amount, Date, Merchant) using AI. 5. The system automatically categorizes the expense based on the extracted data. 6. System redirects to the Results Screen and displays the confirmation form containing all categorized information to the user.

	7. The user reviews the data and clicks the “Save Transaction” button. 8. The system saves the transaction to the database, updates the wallet balance, displays a success message and returns the Receipt Scanning Screen.
Alternative Flows	**Alternative flow 1: System requires Camera permission (First-time use)** 1. From #2 of the basic flow, System detects camera permission is not configured and displays a permission request dialog. 2. The user clicks “Allow”. 3. The system activates the camera stream inside the viewfinder. 4. Continue step #3 in the basic flow. **Alternative flow 2: User denies Camera permission** 1. From #1 of “Alternative flow 1”, User clicks “Don’t Allow”. 2. The system displays an error message prompting for permission, and blacks out the camera viewfinder area on the current screen. 3. Users can now only interact with the “Choose from Gallery” button right on that same screen. 4. Continue step #2 in “Alternative flow 3”. **Alternative flow 3: User chooses to upload receipt from Gallery** 1. At step #2 or step #3 of the basic flow (or from step #3 of “Alternative flow 2”), the user clicks the “Choose from Gallery” button. 2. The system checks gallery permissions. *(If permission is not configured, go to “Alternative flow 3A”)*. 3. The system opens the device’s photo album for the user to pick an existing receipt image. 4. The system uploads the selected image into the processing system. 5. Continue step #4 in the basic flow. **Alternative flow 3A: System requires Gallery permission (First-time use)** 1. From #2 of “Alternative flow 3”, System detects gallery permission is not granted and displays a permission request dialog. 2. The user clicks “Allow”. 3. Continue step #3 in “Alternative flow 3”. **Alternative flow 3B: User denies Gallery permission** 1. From #1 of “Alternative flow 3A”, User clicks “Don’t Allow”. 2. System displays an error message: “Gallery access is required to select photos. Please enable it in Settings or use the Camera.” 3. The user acknowledges the error message.

	4. System returns the user to the Receipt Scanning Screen (step #2 of the basic flow) so they can try again or use the camera if available. **Alternative flow 4: Image is blurry or unrecognizable** 1. From #4 of the basic flow, AI cannot extract data due to poor image quality. 2. System displays an error message: “Cannot recognize image. Please retake or choose another photo.” 3. User acknowledges the error message. 4. System keeps the user on the Receipt Scanning Screen (step #2 of the basic flow) so they can retake the photo or select another one. **Alternative flow 5: Network connection error during AI extraction** 1. From #4 of the basic flow, connection to the AI server is lost or timed out. 2. The system displays an error message: “Network connection lost. Please check your connection and try again.” and displays a “Retry” button. 3. The user clicks the “Retry” button. 4. Continue step #4 in the basic flow. **Alternative flow 6: User corrects incorrect AI data** 1. From #6 of the basic flow, User finds wrong extracted information on the results screen. 2. The user manually inputs the correct values into the corresponding fields. 3. The system registers the newly updated data. 4. Continue step #7 in the basic flow. **Alternative flow 7: User cancels the process at any time** 1. Before step #7 of the basic flow, the user clicks the “Cancel” or “Close” (X) button. 2. The system terminates the process and clears the temporary image cache. 3. The system returns the user to the homepage.
Pre-conditions	The user is logged into the application and connected to the Internet.
Post-conditions	The user successfully saves the new transaction, the wallet balance is updated, and the financial reports are refreshed with new data.

2.14 Use-case: Manage System Dashboard

Use case Name	Manage System Dashboard

Brief description	This use case describes how the Admin monitors specific system analytics and utilizes the dashboard as a central control center to navigate to other administrative modules.
Actors	Admin
Basic Flow	1. Admin clicks on the 'Admin Dashboard' tab from the system navigation menu. 2. System checks the admin privileges of the logged-in account. 3. System displays the Dashboard screen, rendering the following specific data components: - User Metrics: Total registered users, monthly user growth rate, active users (DAU/MAU), and a timeline chart showing user acquisition trends over weeks/months. - AI & Transaction Metrics: Total AI scans, AI success/failure rates, average OCR processing time, and a ratio chart comparing manual input versus AI scanning data. - Financial Operation Costs: Current month's estimated API expenditure for OCR/AI services, and the remaining token/credits quota. - System Alerts Counter: Number of pending user feedbacks and unresolved bug reports waiting for action. - Quick Action Buttons: Direct entry points to ‘User Management’, ‘Notification Center’, ‘AI Logs’, and ‘Feedback Management’. 4. Admin reviews the displayed data components and clicks on one of the Quick Action buttons to perform management tasks. 5. System redirects the Admin to the corresponding dedicated management interface. 6. Admin performs necessary administrative tasks (e.g., blocking a user, sending a push notification, or reviewing AI error logs). 7. Admin clicks ‘Back to Dashboard’ after completing the tasks. 8. System refreshes all the dashboard data metrics listed in Step #3 and displays the updated screen.
Alternative Flows	**Alternative flow 1: User does not have admin privileges** 1. From #2 of the basic flow, System detects the account does not possess admin rights. 2. System blocks access and displays an error message: “Access Denied. You do not have admin permissions.” 3. System redirects the user back to the public application homepage. **Alternative flow 2: Critical API Quota warning triggered** 1. From #3 of the basic flow, System detects the remaining API credits or tokens fall below the 20% safety threshold.

	2. System highlights the Financial Operation Costs section in red and displays an alert: "Critical Warning: API Quota is running low. Please recharge." 3. Admin clicks the 'Recharge Quota' shortcut button directly from the alert. 4. System opens the API provider's configuration page. **Alternative flow 3: Admin exits without managing modules** 1. At step #4 of the basic flow, Admin chooses only to monitor the displayed metrics without clicking any quick action buttons. 2. Admin clicks the 'Exit' button. 3. System returns the admin to the application main menu.
Pre-conditions	Admin is logged into the application using an account with designated Admin privileges.
Post-conditions	The system metrics are updated in real-time, administrative actions are applied, and the system dashboard is refreshed successfully.

2.15 Use-case: Manage User Accounts

Use case Name	Manage User Accounts
Brief description	This use case describes how the Admin browses, searches, views profiles, edits information, manages account statuses, and reviews logs of all registered users in the system.
Actors	Admin
Basic Flow	1. From the Admin Dashboard, Admin clicks on the 'User Management' shortcut button. 2. System checks the admin privileges of the account. 3. System redirects the Admin to the User Management Screen, displaying a list of all registered users with basic registration details and a search/filter bar. 4. Admin enters keywords (Name/Email) into the search bar or applies status filters to find a specific user. 5. System filters and displays the matching user accounts. 6. Admin selects a user account and chooses one of the administrative actions: - View full profile details (Go to Alternative Flow 5) - Edit profile information (Go to Alternative Flow 6). - Activate / Deactivate the account status (Go to Alternative Flow 7). - View user activity logs (Go to Alternative Flow 8).

	7. System processes the selected action and updates the database successfully. 8. Admin clicks the 'Back to Dashboard' button. 9. System returns the Admin to the main Admin Dashboard screen.
Alternative Flows	**Alternative flow 1: User list is empty** 1. From #3 of the basic flow, System detects there are no registered users in the database. 2. System displays a placeholder message: "No users found in the system" 3. Continue step #8 in the basic flow. **Alternative flow 2: User Management page fails to load** 1. From #3 of the basic flow, System encounters a critical database error and cannot load the user list. 2. System displays an error message: "Failed to load user data. Please try again later" and shows a 'Return' button. 3. Admin clicks the 'Return' button. 4. Continue step #9 in the basic flow. **Alternative flow 3: User not found** 1. From #5 of the basic flow, System cannot find any user matching the search criteria. 2. System displays a notice: "No users matching your search keyword" 3. Continue step #4 in the basic flow. **Alternative flow 4: System fails to update account modification** 5. From #7 of the basic flow, System cannot write data to the server due to a connection timeout. 6. System displays an error notification: "Unable to update user information" and provides two options: 'Retry' or 'Cancel'. 7. If Admin chooses 'Retry': Continue step #7 in the basic flow. 8. If Admin chooses 'Cancel': System discards the changes. Continue step #3 in the basic flow. **Alternative flow 5: Admin views full profile details** 1. Admin clicks 'View Details' on the selected user. 2. System opens the user's detailed profile page showing full account information. 3. Admin reviews the profile and clicks 'Close'. 4. Continue step #6 in the basic flow. **Alternative flow 6: Admin edits user profile information**

	1. Admin clicks ‘Edit Profile’ on the selected user. 2. System enables editable fields. 3. Admin modifies the text fields and clicks ‘Save Changes’. 4. Continue step #7 in the basic flow. **Alternative flow 7: Admin activates/deactivates an account** 1. Admin toggles the account status switch. 2. If deactivating: Admin sets status to *Inactive* (User will be blocked from logging in). 3. If activating: Admin sets status to *Active* (Restores user access). 4. Continue step #7 in the basic flow. **Alternative flow 8: Admin views user activity logs** 1. Admin clicks ‘View User Logs’. 2. System fetches and displays the history of actions performed by that user (e.g., date of last login, number of AI scans executed). 3. Admin reviews the logs and clicks ‘Back’. 4. Continue step #6 in the basic flow.
Pre-conditions	Admin is logged into the application using an account with designated Admin privileges.
Post-conditions	The user database is modified according to the admin's actions, and changes are applied to user login permissions immediately.

2.16 Use-case: Manage Push Notifications

Use case Name	Manage Push Notifications
Brief description	This use case describes how the Admin creates, targets, schedules, and broadcasts manual financial push notifications to users.
Actors	Admin
Basic Flow	1. From the Admin Dashboard, Admin clicks on the ‘Notification Center’ shortcut button. 2. System checks the admin privileges of the account. 3. System redirects the Admin to the Notification Management Screen, displaying a log of previously sent campaigns and a ‘Create New Notification’ button. 4. Admin chooses the ‘Create New Notification’ button. 5. Admin selects the target audience group based on expense application filters: - All users

	- Specific financial status groups (e.g., New Users, Active Users, Users Over Budget) 6. Admin selects the delivery schedule: - Send immediately - Schedule for later (e.g., automated daily entry reminders at 9:00 PM) 7. Admin clicks “Send” or “Schedule”. 8. System delivers the push notification and logs the campaign action.
Alternative Flows	**Alternative flow 1: Missing required fields** 1. From #3 of the basic flow, Admin tries to submit the form with a missing title or message content. 2. System blocks the submission and displays the error message: “Title and message content are required.” 3. Continue step #3 in the basic flow. **Alternative flow 2: Firebase Cloud Messaging Server connection failure** 1. From #7 of the basic flow, System fails to broadcast messages due to a Firebase server connection timeout. 2. System displays the error message: “Failed to broadcast notification due to server timeout.”, marks the campaign status as “Failed”, and provides a ‘Retry’ button. 3. If Admin clicks ‘Retry’**:** Continue step #7 in the basic flow. 4. If Admin clicks ‘Cancel’**:** Continue step #3 in the basic flow. **Alternative flow 3: Admin cancels a scheduled campaign** 1. From #1 of the basic flow, Admin selects an existing notification campaign from the log that has a “Scheduled” status instead of creating a new one. 2. Admin clicks “Cancel Schedule” button. 3. System removes the item from the automation queue, updates its status to “Canceled” and refreshes the screen. 4. Continue step #1 in the basic flow.
Pre-conditions	1. Admin is authenticated and logged into the application with appropriate designated Admin privileges. 2. System is accessible and connected to the Internet to sync with the push notification server.
Post-conditions	1. Push notification is delivered to the selected user immediately or queued securely on the server for scheduled delivery. 2. Users see the financial updates or tips in their device's notification center. 3. System logs the notification campaign content, target group, sender,

	timestamp, and final delivery status.

2.17 Use-case: View AI Scan Logs

Use case Name	View AI Scan Logs.
Brief description	Allows Admins to access and review the chronological history of all AI and OCR receipt scanning activities.
Actors	Admin
Basic Flow	1. Admin navigates to the AI Logs interface from the main dashboard shortcut. 2. System fetches and displays a chronological list of recent AI scan activities, including Transaction ID, Status (Success/Failed), and Timestamp. 3. Admin inputs a Transaction ID or filters the list status to find a specific scanning record. 4. System displays the filtered matching scan records on the screen. 5. Admin choose a specific log record to inspect its technical details. 6. System renders the Detailed Log screen, displaying the uploaded raw receipt image, extracted OCR text strings, AI-assigned categories, and error codes (if any). 7. Admin reviews the diagnostic data and clicks the “Close” button to exit the detailed view. 8. System returns the Admin to the main AI Scan Logs list view.
Alternative Flows	**Alternative flow 1: AI scan log history is empty** 1. From #2 of the basic flow, System detects no AI scanning activities have been logged in the database. 2. System displays a placeholder message: “No AI scanning logs recorded in the system.” 3. Continue step #8 in the basic flow. **Alternative flow 2: Database failure to load logs** 5. From #2 of the basic flow, System encounters a severe database connection timeout and cannot fetch the log list. 6. System displays an error banner: “Failed to sync AI logs. Please check network connection.” and provides a ‘Retry’ button. 7. If Admin clicks 'Retry': Continue step #2 in the basic flow. 8. If Admin clicks 'Cancel': System returns the admin to the main Admin Dashboard. **Alternative flow 3: Specific log record not found**

	9. From #4 of the basic flow, System cannot find any log entry matching the searched Transaction ID or applied filters. 10. System displays a notice: “No matching log records found.” 11. Continue step #3 in the basic flow. **Alternative flow 4: Admin corrects AI misclassification (Data Relabeling)** 12. From #6 of the basic flow, Admin notices that the AI auto-categorization algorithm assigned a wrong expense category to the receipt text. 13. Admin clicks the “Relabel Data” button, selects the correct financial category from a dropdown list, and clicks “Save”. 14. System saves the corrected label to the database to serve as future model training data and updates the status to “Resolved”. 15. Continue step #7 in the basic flow.
Pre-conditions	1. Admin is authenticated and logged into the application with assigned Admin management privileges. 2. The application database and AI processing logs are active and accessible.
Post-conditions	1. Admin successfully diagnoses system scanning performance or errors. 2. Any manual data relabeling or adjustments made by the admin are written to the system database for algorithm improvement. 3. System records the admin's inspection activities in the internal audit trailing logs.

2.18 Use-case: Manage User Feedback

Use case Name	Manage User Feedbacks.
Brief description	Allows Admins to access, review, and resolve feedbacks, bug reports, and feature suggestions submitted by users regarding application performance or AI scanning mismatches.
Actors	Admin
Basic Flow	1. Admin navigates to the Feedback Management interface from the main dashboard shortcut. 2. System fetches and displays a list of pending user feedbacks, showing Submission ID, User Email, Category (Bug/Suggestion), and Status (Pending). 3. Admin uses filters to sort by keywords to locate a specific bug report or inquiry. 4. System displays the filtered feedback entries on the screen. 5. Admin chooses a specific feedback row to view details.

	6. System opens the Detailed Feedback screen, displaying the user's text description, uploaded screenshot attachments (if any), and submission timestamp. 7. Admin types a resolution response in the reply text field and updates the status dropdown to “Resolved”. 8. Admin clicks the “Submit Response” button 9. System writes the resolution logs to the database, triggers an automated email response to the user, and refreshes the dashboard counters. 10. Continue step #2 in the basic flow.
Alternative Flows	**Alternative flow 1: Feedback queue is empty** 1. From #2 of the basic flow, System detects no user feedbacks are currently logged in the database. 2. System displays a placeholder message: “No pending user feedbacks found.” 3. Admin clicks the ‘Back to Dashboard’ button. 4. System returns the Admin to the main Admin Dashboard screen. **Alternative flow 2: Database failure to save resolution** 1. From #9 of the basic flow, System experiences a network timeout and fails to update the feedback status. 2. System blocks the action, displays an error alert: “Failed to update feedback status due to connection error.”, and provides ‘Retry’ or ‘Cancel’ options. 3. If Admin clicks ‘Retry’: Continue step #9 in the basic flow. 4. If Admin clicks ‘Cancel’: System discards the response draft. Continue step #6 in the basic flow. **Alternative flow 3: Admin forwards bug report to developers** 1. From #7 of the basic flow, Admin recognizes the feedback is a critical backend system bug instead of a simple user inquiry. 2. Admin clicks the “Forward to Dev Team” button, flags the priority level, and inputs technical notes. 3. System sets the feedback status to “In Progress”, assigns a developer tracking ticket, and updates the history view. 4. Continue step #2 in the basic flow.
Pre-conditions	1. Admin is authenticated and logged into the application with designated Admin management privileges. 2. The user feedback tracking database is functional and accessible online.
Post-conditions	1. User feedbacks are successfully reviewed, resolved, or escalated to appropriate technical pipelines.

	2. Automated confirmation notifications regarding the resolution are dispatched to the reporting users.